



FERIT

Faculty of Electrical Engineering,
Computer Science and Information Technology Osijek
(FERIT Osijek)

Embedded Systems

Peugeot Infotainment Project

Author(s):
Mislav Milinković
Ivan Brajinović

2. rujna 2025.

Academic year 2024/2025

Sadržaj

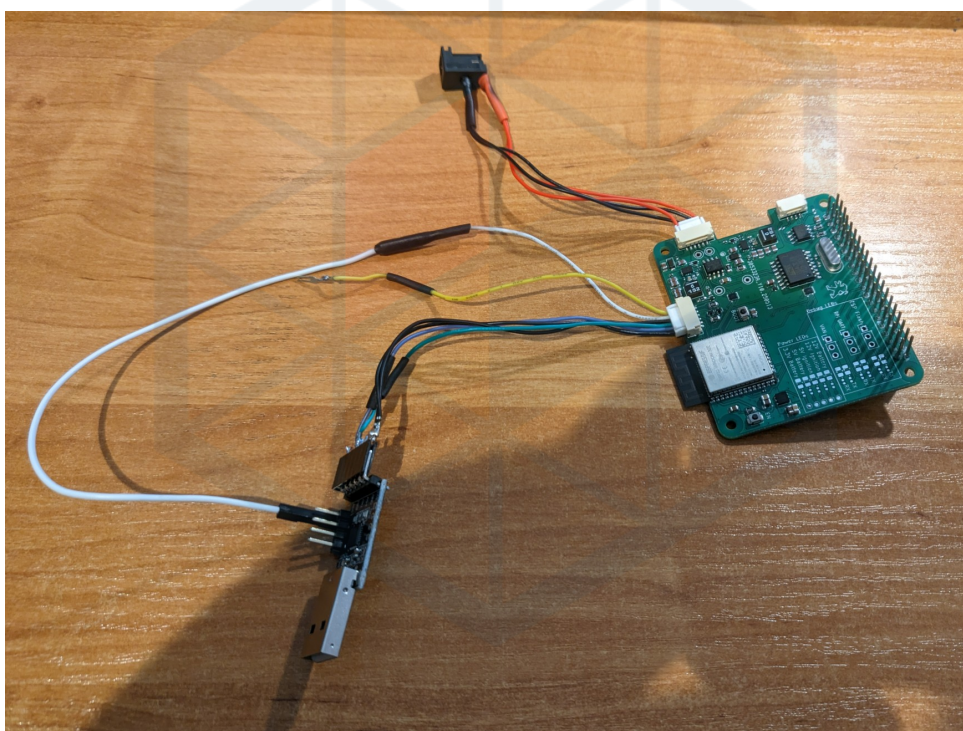
1	Projekt	2
1.1	Uvod	2
1.2	Glavni izazovi	2
2	VAN sabirnica	4
2.1	Unaprijeđeno Manchesterovog kodiranje	4
2.2	Messages	4
2.2.1	Tipovi poruka	6
2.3	VAN mreža	7
3	Sklopovlje pristupnog modula	9
3.1	Hardver za komunikaciju	9
3.1.1	TSS463C	9
3.2	Naponske razine	9
3.3	Povezivanje s vozilom	9
3.4	ESP32 Flashing	10
3.5	Napajanje	11
3.6	Raspberry Pi	11
3.7	Ostalo	11
3.8	Dizajnerske greške	12
4	Gateway module software	13
4.1	Okvir i pristup	13
4.2	Primanje VAN okvira	13
4.3	Analiza VAN okvira	14
4.4	Slanje VAN okvira	14
4.5	UART komunikacija	15
4.6	Upravljanje događajima i zadacima	15
4.7	Upravljanje režimima napajanja	15
4.8	ULP koprocesor	16
5	Infotainment computer software	19
6	Conclusion and future plans	20
	Literatura	21

1 Projekt

1.1 Uvod

Cilj ovog projekta bio je izraditi pristupni modul za informacijsko-zabavni sustav koji će se koristiti u vozilu Peugeot 206. Sam sustav sastoji se od dvaju glavnih dijelova: pristupnog modula i informacijsko-zabavnog računala. Pristupni modul izrađen je kao prilagođena pločica temeljena na modulu ESP32 Wroom 32E. Pločica upravlja napajanjem, komunikacijom s vozilom te komunikacijom s informacijsko-zabavnim računalom. Informacijsko-zabavno računalo čini Raspberry Pi 4 sa zaslonom osjetljivim na dodir (DSI) i prilagođenom Linux distribucijom. Računalo upravlja prikazom slike i zvuka.

Ovaj project je proširenje postojećeg završnog rada ‘VAN protokol i njegova primjena u automobilima’ [5].



Slika 1: Izgled spoja pristupnog modula za svrhe razvoja

1.2 Glavni izazovi

Ovaj projekt ima dva, možda tri glavna izazova koje treba razmotriti. Najizraženiji je komunikacija s vozilom. Peugeot 206 ne koristi standardnu CAN sabirnicu, već nešto što se naziva VAN, vidi poglavlje 2. Taj komunikacijski standard danas je zastario, jer je korišten isključivo u vozilima Peugeot i Citroën tijekom ograničenog vremenskog razdoblja. Zbog toga gotovo da i ne postoji hardver koji može razumjeti taj protokol, a dokumentacije je približno jednako malo, pri čemu je većina napisana na francuskom jeziku. Stoga moramo izraditi vlastiti analizador (parser) za primanje VAN okvira te pronaći način kako ih i slati.

Drugi izazov odnosi se na napajanje. Imamo komponente koje zahtijevaju i 5 volti i 3,3 volta, kao i dvije naponske vodilice: trajni izvor i uključivi izvor. Dodatni problem je **vrlo** „šumno” i ponekad nestabilno ulazno napajanje, pa jeftini istosmjerni pretvarači (DC-DC) neće biti dovoljni. Posljed-

nji izazov, a možda i najpodcijenjeniji, jest izrada pouzdane programske podrške (firmwarea) za sam pristupni modul. Modul ne treba samo analizirati VAN okvire, već mora upravljati vlastitim stanjem, pratiti stanje vozila, upravljati napajanjem i komunikacijom informacijsko-zabavnog računala, kontrolirati režime mirovanja i upravljati vlastitim niskonaponskim stanjem, te, što je najvažnije, ne smije dolaziti do pada sustava.



FERIT

2 VAN sabirnica

VAN (eng. *Vehicle Area Network*) je serijska komunikacijska sabirnica koju su razvili PSA grupa (Peugeot, Citroën) i Renault kao alternativu CAN sabirnici. Definirana je u ISO standardu **ISO 11519-3**. Koristi istu fizičku razinu kao CAN sabirnica (diferencijalni par) i identičnu metodu arbitraže na sabirnici, ali se razlikuje u načinu pakiranja podataka. Gotovo sve informacije navedene ovdje preuzete su iz Atmel TSS463C datasheeta [3].

Kao i CAN, VAN protokol koristi identifikatore za razlikovanje poruka i izvođenje arbitraže na sabirnici. Dodatno, omogućuje tzv. „odgovore unutar okvira” (In-frame), veće veličine poruka (28 bajtova umjesto 8 bajtova kao u CAN-u) te mogućnost izravne komunikacije s drugim uređajem.

Podaci na VAN mreži pakiraju se u okvire. Okviri se sastoje od više dijelova: početka okvira (**SOF**), identifikatora poruke (**IDEN**), naredbe (**COM**), podataka poruke (**DATA**), kontrolnog zbira okvira (**FCS**), signala kraja podataka (**EOD**), signala potvrde (**ACK**) te signala kraja okvira (**EOF**).

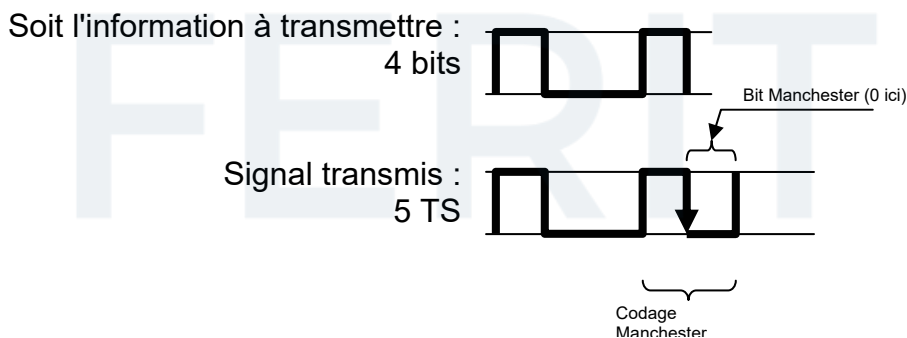
SOF 10 TS	IDEN 12 bit 15 TS	Command (4 bit, 5 TS)				DATA max. 28 byte 280 TS	FCS 15 bit 18 TS	EOD 2 TS	ACK 2 TS	EOF 4 TS
		EXT	RAK	R/W	RTR					

Tablica 1: VAN format okvira

2.1 Unaprijeđeno Manchesterovog kodiranje

Svi VAN okviri kodirani su pomoću unaprijeđenog Manchesterovog kodiranja (eng. *Enhanced Manchester*, *e-Manchester*). Kod e-Manchestera, nakon slanja tri NRZ bita, četvrti bit šalje se kao Manchesterov bit. Slanje podataka na ovaj način omogućuje lakšu sinkronizaciju na sabirnici. Ovo je predvidljivija metoda ubacivanja bitova (bit-stuffing) u usporedbi s metodom korištenom na CAN sabirnici.

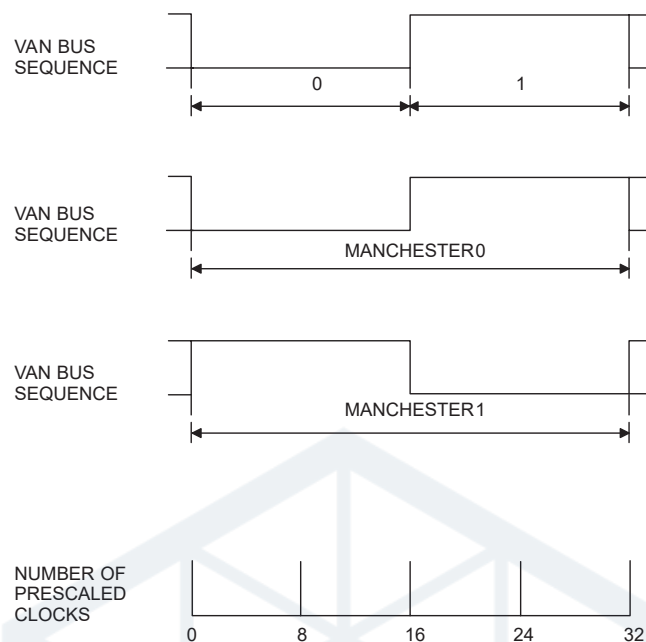
U Tablici 1 prikazan je broj bitova za svaki dio okvira, kao i broj vremenskih intervala (Time Slots, TS). Budući da Manchesterov bit zahtijeva dva vremenska intervala za prijenos, četiri bita u nizu kodirana e-Manchester metodom prenose se u 3 + 2 vremenska intervala.



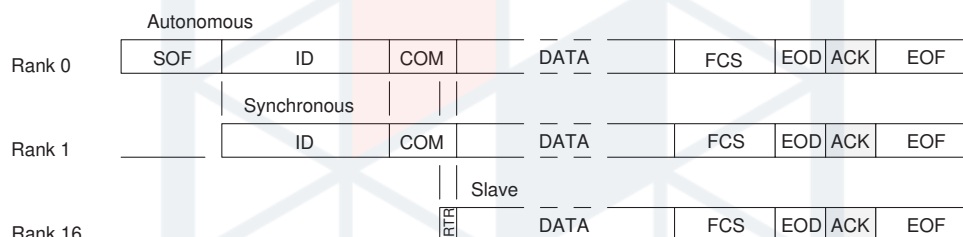
Slika 2: e-Manchester encodiranje [1]

2.2 Messages

Uređaj ili modul povezan na mrežu može biti jednog od tri tipa:



Slika 3: NRZ i Manchester bitovi. [3]



Slika 4: Tipovi modula

- Autonomni modul – glavni uređaj sabirnice. Može slati sekvence **SOF**, inicirati prijenos podataka i primiti poruke.
- Modul sa sinkronim pristupom – ne može slati sekvence **SOF**, ali može inicirati prijenos podataka i primiti poruke.
- Podređeni modul (eng. *Slave*) – može slati poruke samo pomoću odgovora unutar okvira (eng. *in-frame*), a također može primiti poruke.

VAN protokol podržava više tipova poruka. Razlikuju se pomoću polja identifikatora (**IDEN**) i naredbe (**COM**). Identifikator je dug 12 bita i slijedi ga 4 bita naredbe. Zbog načina na koji funkcionira arbitraža na sabirnici, manji identifikator ima prednost na sabirnici. Dok je vrijednost identifikatora potpuno proizvoljna, bitovi naredbe su strogo definirani i određuju tip poruke koja će se poslati na sabirnici:

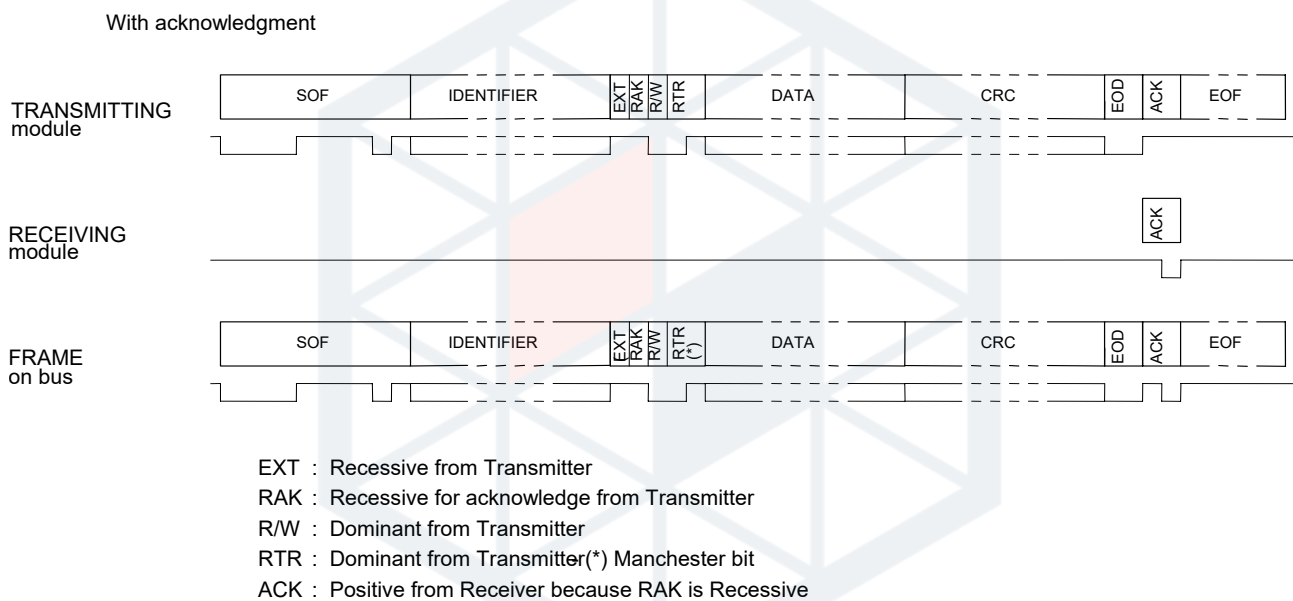
- EXT – uvijek 1
- RAK – „Request Acknowledge” (zahtjev za potvrdom). Ako je 1, primatelj mora potvrditi primitak poruke. Ako je 0, signal ACK se ne šalje.
- R/W – „Read/Write” (čitanje/pisanje). Označava smjer podataka. Ako je 0, radi se o poruci za pisanje („write”), gdje uređaj šalje podatke namijenjene drugom uređaju. Ako je 1, radi se o poruci za čitanje („read”), što podrazumijeva zahtjev za podacima i očekuje odgovor.

- RTR – „Remote Transmission Request” (zahtjev za daljinskim prijenosom). Ako je 0, poruka sadrži podatke. Ako je 1, poruka ne sadrži podatke i implicira zahtjev za odgovor unutar okvira (in-frame response). Kombinacija R/W = 0 i RTR = 1 je zabranjena na sabirnici.

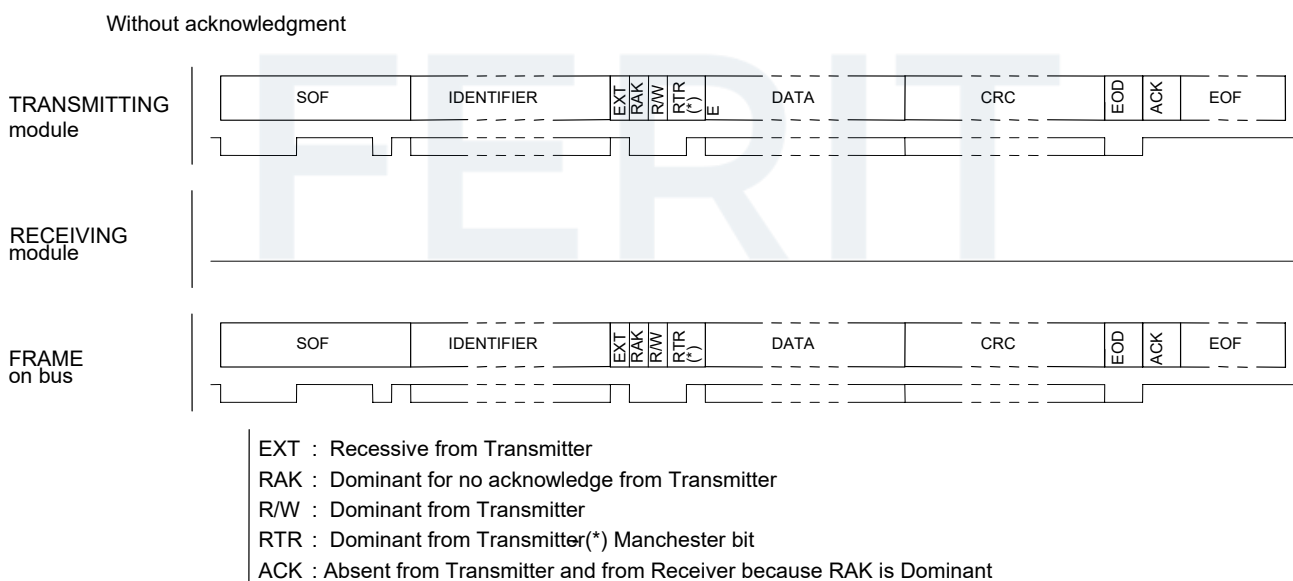
2.2.1 Tipovi poruka

Ovdje ćemo navesti tipove okvira relevantne za ovaj projekt. Postoje i drugi, ali koristimo samo ove. Podsjetnik: Dominantno = 0, Recesivno = 1.

Normalni okvir Najjednostavniji tip okvira. Jedan modul šalje cijeli okvir s podacima i može, ali ne mora, zatražiti potvrdu (ACK). U slučaju da se potvrda ne traži, poruka se tretira kao poruka za emitiranje (broadcast).

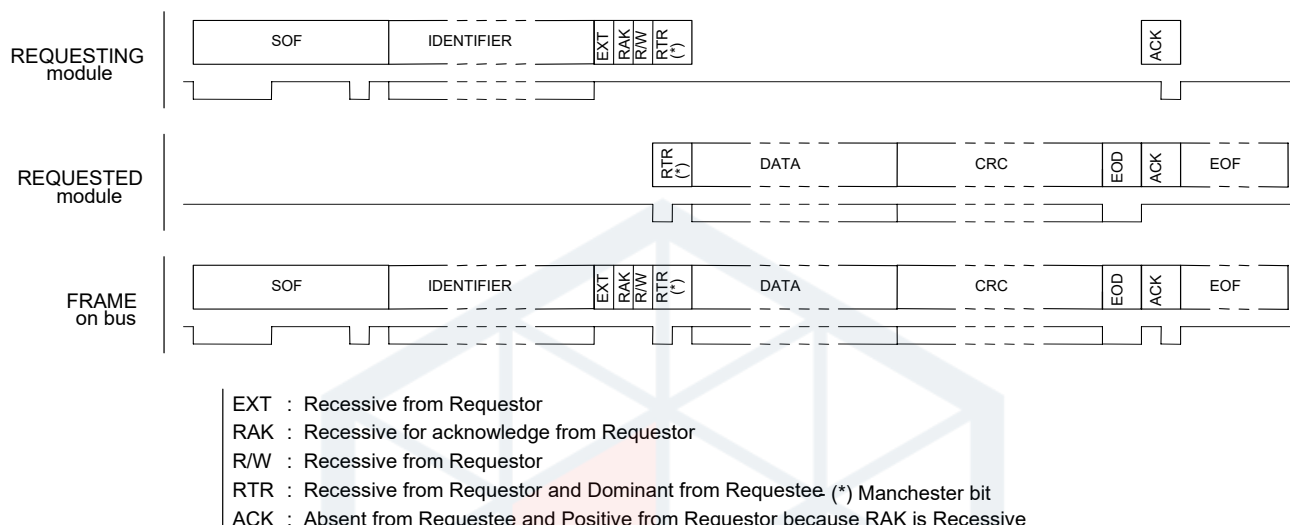


Slika 5: Prijenos normalnog okvira sa ACK



Slika 6: Prijenos normalnog okvira bez ACK

Okvir zahtjeva za odgovor s neposrednim odgovorom Ovo je jedini okvir u kojem podređeni modul (slave) može slati podatke. Jedina razlika na sabirnici je promjena stanja bita R/W u odnosu na normalni okvir. Glavni modul sabirnice (bus master) generira **SOF**, inicijator generira identifikator i prva tri bita naredbe, te signal ACK. Ostalo (RTR, podaci i kontrolni zbir) generira modul koji odgovara.

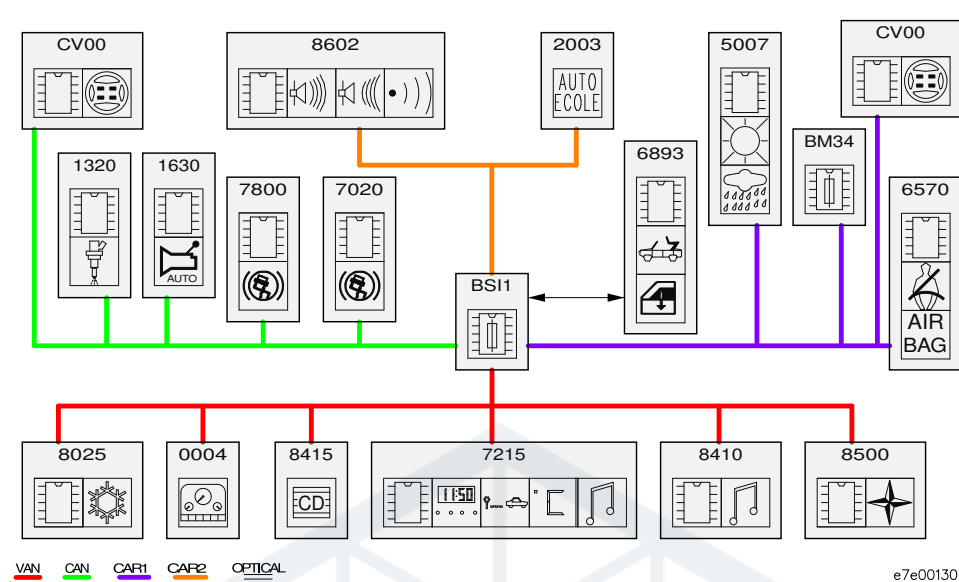


Slika 7: Okvir zahtjeva za odgovor s neposrednim odgovorom

2.3 VAN mreža

Više modula možemo povezati zajedno kako bismo stvorili mrežu. Također je moguće povezati više mreža i koristiti pristupni modul (gateway) za usmjeravanje podataka između mreža. U vozilu Peugeot 206 imamo jednu CAN mrežu *CAN Engine* i tri VAN mreže: *VAN Comfort*, *VAN Body 1* i *VAN Body 2*.

Raspored mreža prikazan je na slici 8. Tamo također možemo vidjeti pristupni modul, BSI. Ovaj modul prisutan je u svim vozilima PSA grupe. Konkretni BSI dolazi iz generacije AEE2001, jedine generacije koja koristi VAN sabirnicu. Druge generacije, AEE2004 i AEE2010, koriste isključivo CAN. Naš informacijsko-zabavni sustav bit će spojen na mjesto CD mijenjača, modul broj 8415 na slici 8 na *VAN Comfort* sabirnici.



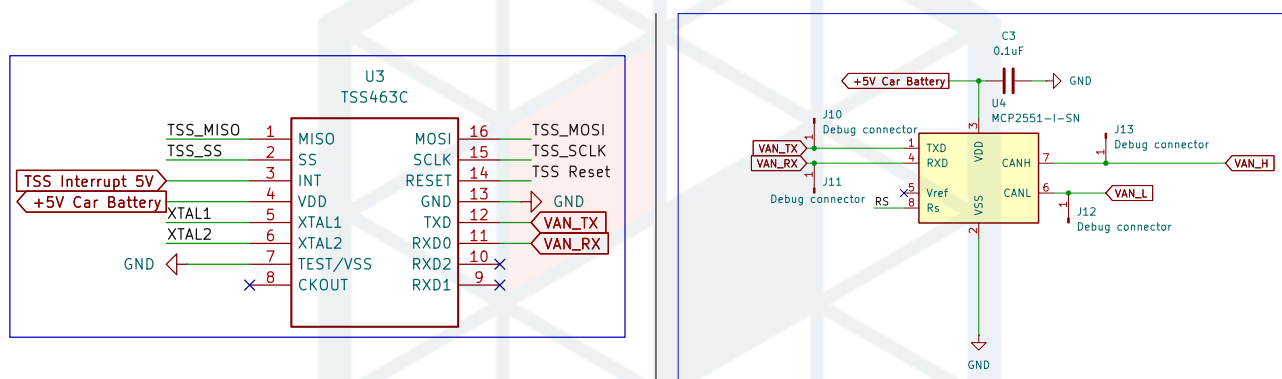
Slika 8: VAN and CAN Network layout in the Peugeot 206 [2]

3 Sklopovlje pristupnog modula

Pristupni modul dizajniran je kao Raspberry Pi HAT ploča. Mora sadržavati potrebnu sklopovsku podršku za regulaciju napajanja i komunikaciju s vozilom. Također mora pružiti sve ulaze/izlaze (IO) za povezivanje vanjskih uređaja, poput programatora za ESP32 i napojnog kabela za zaslon Raspberry Pi-ja.

3.1 Hardver za komunikaciju

U poglavlju 2 utvrdili smo da je VAN sabirnica diferencijalni protokol, slično CAN sabirnici, te da se stoga može koristiti već dostupni hardver za CAN prijemnike i odašiljače. Ipak, još uvijek nam je potreban hardver koji može generirati valjane VAN okvire. Za to koristimo Atmel TSS463C VAN upravljački čip. To je jedini postojeći VAN upravljački čip. Za CAN prijemnik koristimo MCP2551 jer je dokazano da radi s ranim prototipovima, a sa svojom logikom od 5 volti dobro se integrira s TSS čipom.



Slika 9: TSS463C and MCP2551 in the schematic

3.1.1 TSS463C

TSS463C je VAN upravljački čip koji koristimo za slanje VAN okvira prema vozilu. To je jedini samostalni VAN upravljački čip koji postoji, jer svi ostali moduli unutar vozila imaju VAN upravljački čip ugrađen u svoje SOC-ove. TSS komunicira s ESP32 preko SPI sučelja.

3.2 Naponske razine

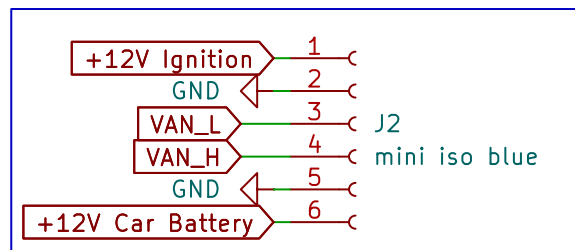
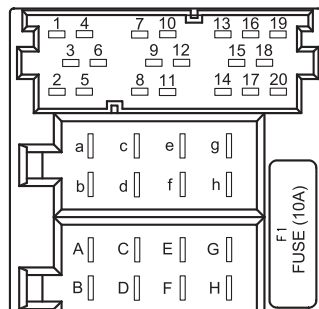
TSS463C i MCP2551 koriste logiku od 5 V, dok svi ostali dijelovi na pločici, uključujući ESP32, koriste logiku od 3,3 V. Za to koristimo par NTS0104 pretvarača naponskih razina. Jedan se koristi za SPI komunikaciju između TSS-a i ESP-a, dok se drugi koristi za povezivanje VAN RX linije s MCP-a na ESP32 radi čitanja VAN okvira, te TSS prekidne linije (interrupt pin).

3.3 Povezivanje s vozilom

Povezujemo se s vozilom na mjestu jedinice za CD mijenjač. CD mijenjač se povezuje s originalnom RD3 glavnom jedinicom u C3 (pinovi 13 do 20) dijela ISO 10487 radio konektora. Pogledajte Sliku 10 za raspored pinova.

Pinovi 13 do 17, koji služe za podatke i napajanje, povezani su s pristupnim modulom pomoću 6-pinskog JST GH konektora, dok su pinovi 18 do 20 spojeni na 3,5 mm audio kabel i povezani s Raspberry Pi-jem. Budući da konvencija imenovanja razlikuje Slike 10 i 11, pogledajte ...

No.	Description
1	N.C.
2	N.C.
3	N.C.
4	N.C.
5	N.C.
6	N.C.
7	VOICE SYN
8	VOICE SYN-GND
9	N.C.
10	N.C.
11	AUX(+)
12	AUX(-)
13	VAN DATA
14	VAN DATA/
15	CD A/C GND
16	CD A/C B/U
17	+VAN
18	CD A/C S-GND
19	CD A/C L
20	CD A/C R



Slika 11: JST Connector in the schematic

Slika 10: ISO connector pinout [4]

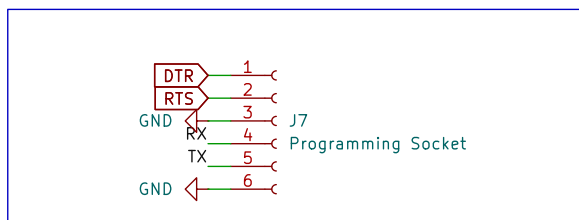
VAN DATA	⇔	VAN_L
VAN DATA/	⇔	VAN_H
CD A/C GND	⇔	GND
CD A/C B/U	⇔	+12V Car Battery
+VAN	⇔	+12V Ignition

Tablica 2: ISO to JST connection legend

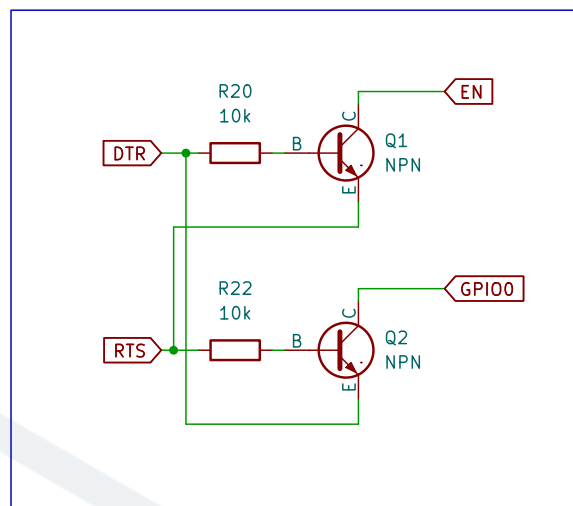
3.4 ESP32 Flashing

Flashing firmwarea na ESP32 obavlja se putem UART sučelja. Kako bi se firmware mogao flashati, ESP se prvo mora staviti u režim flashanja. To se može postići tako da se pinovi GPIO00 i GPIO02 povuku na nisku razinu prilikom uključivanja ploče. Na našoj ploči to se može ostvariti na dva načina: ručno pritiskom na momentalni gumb BOOT dok se ploča resetira pritiskom na gumb EN, ili spajanjem signala DTR i RTS UART adaptera na odgovarajuće pinove na JST programskom konektoru. Oni su interno povezani s ulazima EN i GPIO00, a ESP programator (`esptool.py`) podržava resetiranje i flashanje ploče korištenjem tih signala.

UART programsko sučelje dostupno je na drugom JST GH 6-pinskom konektoru, uključujući signale DTR i RTS.



Slika 12: Programming header pinout



Slika 13: DTR and RTS signals

3.5 Napajanje

Za napajanje imamo dvije 12V naponske vodilice. Prva je uvijek aktivna (+12V Baterija vozila), dok se druga uključuje samo kada je vozilo aktivno (+12V Paljenje). Druga vodilica je pogrešno označena jer ne prati točno stanje ključa paljenja, već stanje aktivnosti vozila. Vozilo se aktivira kada se dogodi neka radnja (zaključavanje/otključavanje, otvaranje/zatvaranje vrata, ključ u ACC položaju itd.) i prelazi u stanje mirovanja nakon određenog vremena ako se ništa ne događa i ključ nije u ACC položaju.

Kako imamo dvije vodilice, imamo dva zasebna regulatora napona (MP8759GD1) koja pretvaraju 12 V ulaza u 5 V izlaza. Regulator na vodilici +12V Baterija napaja ESP32, TSS i MCP čipove, jer oni moraju biti aktivni i kada vozilo spava. Budući da ESP32 zahtijeva 3,3 V, dizajn je proširen jednim LDO regulatorom od 3,3 V.

Drugi 5 V regulator napaja Raspberry Pi i periferni LCD. Kako oni nisu uključeni u komunikaciju s vozilom, nije im potrebno stalno napajanje.

Osim IC regulatora napona, na shemi se nalazi velik broj filter-kondenzatora. Oni su potrebni za stabilan izvor napona za ostale IC-e, budući da su neki vrlo osjetljivi na šum i mogu se početi ponašati nepredvidivo. Još jedan često zanemaren element pri dizajniranju napajanja je TVS dioda, spojena na ulaz regulatora u obrnuto polariziranom načinu, osiguravajući maksimalni napon od 12 V.

3.6 Raspberry Pi

Raspberry Pi je podređeni uređaj ESP32-u u smislu da ne upravlja komunikacijom s vozilom niti bilo kojom drugom kritičnom funkcijom sustava. Njegova uloga je prikaz telemetry podataka primljenih od ESP32 preko UART-a na LCD-u i upravljanje dijelom sustava vezanim uz zabavu. Kao nekritični dio sustava, ne zahtijeva neprekidno napajanje; stoga se napaja samo kada je vozilo aktivno prema tablici 3.

3.7 Ostalo

RTC Budući da postojeći zaslon vozila ne prikazuje vrijeme preko sabirnice, potreban je dodatni RTC. Odabrani RTC je DS3231MZ. Povezan je s Raspberry Pi-jem putem I2C sučelja. Lako je dos-

tupan, precizan i podržan je izravno u Linux kernelu.

Za očitavanje napona baterije pomoću ESP32 ADC-a korišten je jednostavan naponni djelitelj s otpornicima. S omjerom dijeljenja 4:1, na pinu 9 (IO33) dolazi 1/5 napona baterije. Otpornici su oko 100 k Ω , osiguravajući struju u rasponu od 10 μ A.

Za olakšavanje ispravljanja pogrešaka, na komunikacijskim i napojnim linijama postavljene su mnoge LED diode, kao i nekoliko debug konektora.

3.8 Dizajnerske greške

Naravno, projekt nije pravi ako sve radi prvi put. PCB ima nekoliko dizajnerskih grešaka koje su uspješno zaobiđene. Ovdje ćemo navesti neke od njih.

Strapping pinovi Pogriješili smo i previdjeli neke ESP32 strapping pinove. To su pinovi koje ESP koristi pri pokretanju za postavljanje određenih parametara. Pin koji smo previdjeli bio je GPIO12. Taj pin postavlja razinu napona unutarne SPI flash memorije na kojoj je pohranjen firmware. Pin je dijeljen s SPI sabirnicom koja se koristi za komunikaciju s TSS-om, te je stanje pina bilo neodređeno. To je uzrokovalo probleme s pokretanjem i flashanjem ploče. Problem je zaobiđen trajnim postavljanjem razine napona putem eFuse-a.

Reset pinovi MCP i TSS MCP2551 može se staviti u stanje niske potrošnje putem Rs pina. Planirali smo kontrolirati stanje pina pomoću tranzistorskim prekidačem. Međutim, prekidač je bio pogrešno dizajniran i MCP je uvijek bio u niskoj potrošnji, što je sprječavalo slanje podataka na VAN sabirnicu. Problem je zaobiđen trajnim postavljanjem MCP-a u aktivno stanje pomoću 10 k Ω pull-down otpornika, što je povećalo potrošnju u praznom hodu za nekoliko mA.

Isti problem bio je s reset pinom na TSS463C. Budući da se TSS uvijek resetira softverski pri pokretanju, problem je jednostavno zaobiđen uklanjanjem svih komponenti oko reset pina.

Pretvarač naponskih razina Pretvarač naponskih razina korišten za VAN RX liniju spojen na ESP32 koristio je +5V paljenje vodilicu kao referencu za 5 V stranu. Ta vodilica, međutim, bila je isključena kada je ESP odlučio da Raspberry Pi ne treba biti napajan. Kao rezultat, ESP bi ga isključio i više nikada ne bi primio VAN okvir, sprječavajući firmware da ponovno uključi vodilicu. Problem je zaobiđen rezanjem traga i spajanjem reference na +5V Baterija vodilicu.

4 Gateway module software

Pristupni modul je najvažniji dio ovog projekta jer se sučeljava s vozilom i odgovoran je za upravljanje napajanjem cijelog sustava. Modul prima i analizira VAN okvire, organizira podatke i zatim ih proslijeđuje informacijsko-zabavnom računalu. Također obavlja i obrnuti proces: može primiti podatke s informacijsko-zabavnog računala i prenijeti odgovarajući VAN okvir na sabirnicu. Upravljanje napajanjem također je važan dio softvera, budući da prekomjerna potrošnja u praznom hodu brzo iscrpljuje bateriju vozila.

4.1 Okvir i pristup

Za softver koristimo ESP-IDF okvir koji pruža Espressif. Okvir se temelji na operativnom sustavu FreeRTOS. Zbog prirode OS-a i softverskog okruženja, softver modula napisan je koristeći pristup temeljen na događajima.

Koristimo sljedeće događaje kao osnovu za glavne petlje/zadataka:

- Primanje VAN okvira
- Primanje UART podataka s računala
- Promjena stanja vozila
- Timer događaji
- Glavni zadatak

Budući da ESP32 ima dva jezgra, jedno je jezgro posvećeno isključivo primanju i analiziranju VAN okvira, jer su oni vrlo učestali i ne želimo propustiti niti jedan.

4.2 Primanje VAN okvira

Za primanje VAN okvira potrebno je ručno čitati i vremenski pratiti stanja RX pina. Srećom, ESP32 ima hardverski periferyl koji upravo to omogućuje: RMT periferyl. On može pratiti stanja GPIO pinova i mjeriti koliko dugo su u tom stanju. Nakon što postavimo pragove za minimalnu i maksimalnu duljinu signala, periferyl može generirati prekid (interrupt) svaki put kada detektira sekvencu **EOF**.

```
1  rmt_rx_channel_config_t rx_chan_config = {0};
2
3  ...
4
5  timeslot_interval_ns = 8 * 1000; // At 125 kTS/s
6
7  // EOF is 10 TS long
8  rx_rcv_config.signal_range_max_ns = 10 * timeslot_interval_ns;
9  rx_rcv_config.signal_range_min_ns = timeslot_interval_ns / 2;
10
11  ...
```

Listing 1: RMT timing initialization

Kada se detektira sekvenca **EOF**, RMT periferylalni drajver generira prekid (interrupt). U okviru tog prekida, u red zadataka (task queue) modula za primanje VAN okvira postavljamo novi događaj i

ponovno pokrećemo periferijal. Modul za primanje VAN okvira preuzima primljene RMT podatke iz reda i analizira ih. Podaci su polje struktura čija definicija je prikazana u Listingu 2.

VAN okvir rekonstruiramo bit po bit iz tog polja, uzimajući u obzir Manchesterove bitove. Nakon provjere kontrolnog zbira (checksum), izdvajamo identifikator i podatke, stavljamo ih u novi međuspremnik i postavljamo u red glavnog zadatka (Main task) za daljnju obradu. Logika primanja nadahnutu je Peter Pinterovom Arduino bibliotekom [6] i prilagođena je našoj firmware arhitekturi.

```
1  /**
2   * @brief The layout of RMT symbol stored in memory,
3   *       which is decided by the hardware design
4   */
5  typedef union {
6      struct {
7          uint16_t duration0 : 15; /*!< Duration of level0 */
8          uint16_t level0 : 1;    /*!< Level of the first part */
9          uint16_t duration1 : 15; /*!< Duration of level1 */
10         uint16_t level1 : 1;     /*!< Level of the second part */
11     };
12     uint32_t val; /*!< Equivalent unsigned value for the RMT symbol */
13 } rmt_symbol_word_t;
```

Listing 2: Definiton of the RMT data symbol in ESP-IDF

4.3 Analiza VAN okvira

Kada modul za primanje VAN okvira postavi novi događaj, glavni zadatak (Main task) poziva parser paketa koji provjerava identifikator i, ako je prepoznat, analizira podatke u globalne međuspremnike te postavlja odgovarajuće događaje u red glavnog zadatka ovisno o paketu koji se obrađuje. Pogledajte Listing 3 za primjer strukture VAN okvira. Velika većina poznatih dekodiranih VAN okvira dostupna je na web stranici Petera Pintera [7].

```
1  /**
2   * @brief VAN rpm and speed data struct. Iden 0x824
3   *
4   */
5  struct psa_van_rpm_speed_struct
6  {
7      uint16_t rpm;           // RPM in big endian. Divide by 8 to get actual rpm;
8      uint16_t speed;         // Speed in big endian. Divide by 100 to get actual speed;
9      uint16_t distance;      // Distance covered. Increments with each passed decimeter
10     uint8_t packet_changed; // Increments each time information significantly changes
11 } __attribute__((packed));
```

Listing 3: Example VAN frame struct.

4.4 Slanje VAN okvira

Za slanje VAN okvira prema vozilu koristimo TSS463C. TSS je zapravo dodatnih 128 bajtova RAM-a koje moramo upravljati, budući da moramo kontrolirati svu memoriju korištenu za slanje i primanje podataka. Trenutno se TSS koristi samo za slanje paketa CD mijenjača kako bismo ga mogli emulirati,

te za slanje zahtjeva za resetiranje putnog računala. Poruke CD mijenjača su poruke s neposrednim odgovorom (vidi 2.2.1), dok je zahtjev za resetiranje putnog računala normalni okvir s zahtjevom za potvrdom (ACK). Paketi CD mijenjača šalju se periodično, svake sekunde, inače će ugrađeni radio smatrati da CD mijenjač nije prisutan i onemogućiti ga.

TSS komunicira preko SPI sučelja i malo je nezgrapnan za korištenje, pa je napisan prilagođeni drajver radi lakšeg rukovanja. Osim toga, postoji zaseban zadatak koji nadzire stanje TSS poruka i stavlja ih u red za slanje.

4.5 UART komunikacija

Informacijsko-zabavno računalo komunicira s ESP32 preko UART sučelja. Razlog korištenja UART-a je njegova jednostavnost. ESP32 uglavnom šalje podatke računalu, ali također može primiti zahtjeve s računala.

UART kod živi u vlastitom zadatku koji prima događaje iz drugih zadataka i također može slati događaje glavnom zadatku. Kada glavna petlja (Main loop) detektira promjenu u globalnim međuspremnicima podataka (primljen VAN okvir, promjena napona baterije itd.), postavlja odgovarajući događaj u UART zadatak s podacima koji se šalju računalu. Protokol i strukture podataka su prilagođene i inspirirane MSP protokolom korištenim u firmwareu kontrolera leta na FPV dronovima.

4.6 Upravljanje događajima i zadacima

Firmware koristi nekoliko zadataka koji se izvode u vlastitim petljama i čekaju događaje iz više redova. Ukupno postoje 4 globalna reda u firmwareu, po jedan za svaku glavnu komponentu:

- Main Queue – Čita ga glavni zadatak. Svi ostali zadaci šalju svoje događaje u red glavnog zadatka. Glavni zadatak zatim odlučuje što učiniti i gdje proslijediti događaj.
- TSS Queue – Čita ga TSS zadatak. Koristi se za stavljanje paketa u red za slanje od strane TSS-a.
- UART Queue – Čita ga UART zadatak. Koristi se za slanje paketa informacijsko-zabavnom računalu.
- VAN Queue (Neiskorišteno) – Čita ga VAN RMT zadatak. Ostavljen za buduću upotrebu.

Osim toga, postoji interni red u VAN RMT zadatku. Taj red koristi se isključivo između prekida (interrupt) RMT periferalja i VAN RMT zadatka.

Događaji su definirani kao struktura (Listing 4) s identifikatorom događaja (Listing 5) i pokazivačem na podatke. Podaci ovise o primljenom događaju.

4.7 Upravljanje režimima napajanja

Važno je upravljati potrošnjom energije sustava. Radi lakšeg upravljanja definirali smo nekoliko „stanja vozila”. Pogledajte tablicu 3 za popis tih stanja i ponašanje sustava. Jedna važna komponenta spomenuta u tablici je duboki režim spavanja ESP32-a (deep sleep). Kada je u dubokom snu, ESP je praktički isključen. Jedino što radi na čipu je posebni dio RAM-a, RTC periferal i, po potrebi, ULP koprocesor. U našem slučaju koristimo isti ADC pin koji se koristi za mjerenje napona baterije kako bismo probudili ESP. Zbog naponskog raspona pina, ne možemo koristiti jednostavan GPIO wake-up, već moramo koristiti ULP koprocesor za mjerenje napona i buđenje ESP-a kada napon prijeđe određeni prag.

4.8 ULP koprocessor

ESP32 ima vrlo niskopotrošni koprocessor. Ima vrlo jednostavnu arhitekturu i troši vrlo malo energije. Idealno je koristiti ga za ADC mjerenja i buđenje ESP-a. Ne može se programirati izravno u C-u, ali postoje dva načina pisanja koda. Prvi način je pisanje assembler koda u zasebnoj datoteci i njegovo integriranje u firmware binarnu datoteku. Drugi način je korištenje unaprijed definiranih C makroa i pisanje programa u polju unutar C programa te njegovo učitavanje na taj način. Odabrali smo drugu opciju jer je znatno jednostavnija. Pogledajte Listing 6 za ULP kod korišten u dubokom snu.

Description	Computer state	ESP32 state
Car in sleep mode (switched 12V rail is off)	Turned Off (no power)	Deep sleep
Car off and locked, not in sleep mode	Turned Off	Running
Car off, not locked, not in sleep mode	If coming from states above, turned off, otherwise display off	Running
Car off, radio turned on	Turned on, display on	Running
Car off, key in ACC	Turned on, display on	Running
Car off, key in IGN	Turned on, display on	Running
Car cranking	Turned on, display off	Running
Engine running	Turned on, display on	Running

Tablica 3: Car state table

```
1 struct mive_global_event
2 {
3     enum mive_global_event_t event;
4     void* ev_data;
5 };
```

Listing 4: Event struct definition

```

1  enum mive_global_event_t
2  {
3      MIVE_EVENT_NONE = 0,
4      MIVE_EVENT_GLOBAL_SLEEP,
5      MIVE_EVENT_RMT_NEW_VAN_FRAME,
6      MIVE_EVENT_TSS_RESET,
7      MIVE_EVENT_TSS_ACTIVATE,
8      MIVE_EVENT_TSS_IDLE,
9      MIVE_EVENT_TSS_SLEEP,
10     MIVE_EVENT_TSS_WRITE_FRAME,
11     MIVE_EVENT_TSS_MONITOR_CHANNEL,
12
13     MIVE_EVENT_VAN_UPDATE_AUDIO_MENU,
14     MIVE_EVENT_VAN_NEXT_AUDIO_MENU_ITEM,
15     MIVE_EVENT_VAN_CLOSE_AUDIO_MENU,
16
17     MIVE_EVENT_UART_NEW_DATA, // New VAN Data to send
18     MIVE_EVENT_UART_RECEIVE, // Received data from UART
19     MIVE_EVENT_UART_SEND, // Send data to UART
20
21     MIVE_EVENT_TIMER_UART_SEND,
22     MIVE_EVENT_TIMER_1MS,
23     MIVE_EVENT_TIMER_1S,
24
25     MIVE_EVENT_ADC,
26     MIVE_EVENT_STATE_CHANGE,
27     MIVE_EVENT_POWER,
28 };

```

Listing 5: List of some events present in the firmware

```

1  const ulp_insn_t program[] = {
2      I_MOVI(R0, 0), // Set reg. R0 to initial 0
3      I_MOVI(R2, 0), // Set reg. R2 to initial 0
4      M_LABEL(1), // LABEL 1 - Start of measurement
5      I_ADDI(R0, R0, 1), // Increment cycle counter (reg. R0)
6      I_ADC(R1, PSA_ADC_UNIT, PSA_ADC_CHANNEL), // Read ADC value to R1
7      I_ADDR(R2, R2, R1), // Add ADC value from R1 to R2
8      M_BL(1, 4), // If cycle counter is less than 4, go to LABEL 1
9      I_RSHI(R0, R2, 2), // Divide accumulated ADC value in R2 by 4 and save it t
10     M_BGE(2, high_adc_treshold), // If average ADC value from reg.
11     // R0 is higher or equal than high_adc_treshold, go to L
12     I_HALT(), // Halt the coprocessor, it will be woken up by the time
13     M_LABEL(2), // LABEL 2
14     I_WAKE(), // Wake up ESP32
15     I_END(), // Stop ULP program timer
16     I_HALT(), // Halt the coprocessor
17 };
18
19 size_t size = sizeof(program)/sizeof(ulp_insn_t);
20 ESP_ERROR_CHECK(ulp_process_macros_and_load(0, program, &size));
21 ulp_set_wakeup_period(0, 20000); // ULP wakeup timer
22 err = ulp_run(0);

```

Listing 6: ULP code that runs in deep sleep.

```
1 int ret = xQueueReceive(main_queue, &event, pdMS_TO_TICKS(1000));
2
3 if(ret == pdPASS)
4 {
5     switch (event.event)
6     {
7         case MIVE_EVENT_RMT_NEW_VAN_FRAME:
8             van_packet = (mive_van_packet_t*)event.ev_data;
9
10            ret = psa_parse_van_packet(
11                van_packet->iden,
12                van_packet->packet_size,
13                van_packet->packet,
14                &global_libpsa_buffers);
15
16            ...
17            break;
18        }
19    }
```

Listing 7: Example event handling

FERIT

5 Infotainment computer software

Informacijsko-zabavno računalo je Raspberry Pi 4 s DSI touchscreen zaslonom. Pokreće prilagođenu Linux distribuciju izrađenu pomoću Yocto build sustava. Grafička aplikacija napisana je koristeći C++ i Qt okvir. Ovaj softver nije glavni fokus ovog rada i neće biti obrađivan detaljno.



Slika 14: Screenshot of an early version of the software.

Značajke:

- Osnovna telemetrija u stvarnom vremenu (broj okretaja motora i temperatura, brzina vozila, status goriva)
- Informacije o vozilu (status vrata, putni računar)
- Informacije o radiju (informacije o stanici, informacije o CD playeru)
- Lokalni glazbeni player
- Bluetooth player
- Kontrola svjetline zaslona ovisno o dobu dana

6 Conclusion and future plans

Tijekom dizajniranja i rada na ovom projektu mnogo smo naučili o unutarnjem funkcioniranju ECU-a vozila, komunikaciji i dizajnu PCB-a. Projekt je pružio jedinstvene izazove, od kojih je najveći bio suočavanje s manje poznatim i dokumentiranim (i to uglavnom na francuskom) VAN protokolom. Već smo započeli rad na sljedećoj reviziji ploče, kao i na sljedećem koraku ovog projekta. Budući da koristimo postojeći zaslon u vozilu, sljedeći korak je reverzno inženjerstvo logike zaslona i njegovo potpuno zamjenjivanje. To je, međutim, tema za neki drugi put.

Još jedan izazov koji je trebalo riješiti u ovom projektu, a koji nije bio toliko spominjan, jest izrada prilagođene Linux distribucije za Raspberry Pi. O tome će biti više riječi u sljedećoj iteraciji projekta.



Slika 15: Infotainment system placed in the car

FERIT

Literatura

- [1] Alain Chautar. „Info Tech n°6”. *Educauto.org* (2003.). url: <http://graham.auld.me.uk/projects/vanbus/datasheets/Mux1.pdf>.
- [2] PSA Peugeot Citroën. *Peugeot Service Box backup - SEDRE*.
- [3] Atmel Corporation. *VAN Data Link Controller with Serial Interface, TSS463C*. 2006.
- [4] Clarion Co. Ltd. *RD3 Service Manual*.
- [5] Mislav Milinković. „VAN protokol i njegova primjena u automobilima”. Završni rad. Fakultet primijenjene matematike i informatike, 2024. url: <https://urn.nsk.hr/urn:nbn:hr:126:246734>.
- [6] Peter Pinter. *ESP32 RMT peripheral Vehicle Area Network (VAN bus) reader*. url: https://github.com/morcibacsi/esp32_rmt_van_rx.
- [7] Peter Pinter. *PSA VAN bus, All data related to Peugeot 206 307 406 S2, Citroen C3 2004*. url: <http://pinterpeti.hu/psavanbus/PSA-VAN.html>.

FERIT